

■ Réalisé par :

Manar BOUAMAR & Amine ANDAME .

■ Encadré par:

M.Khalid BOUKHDIR.

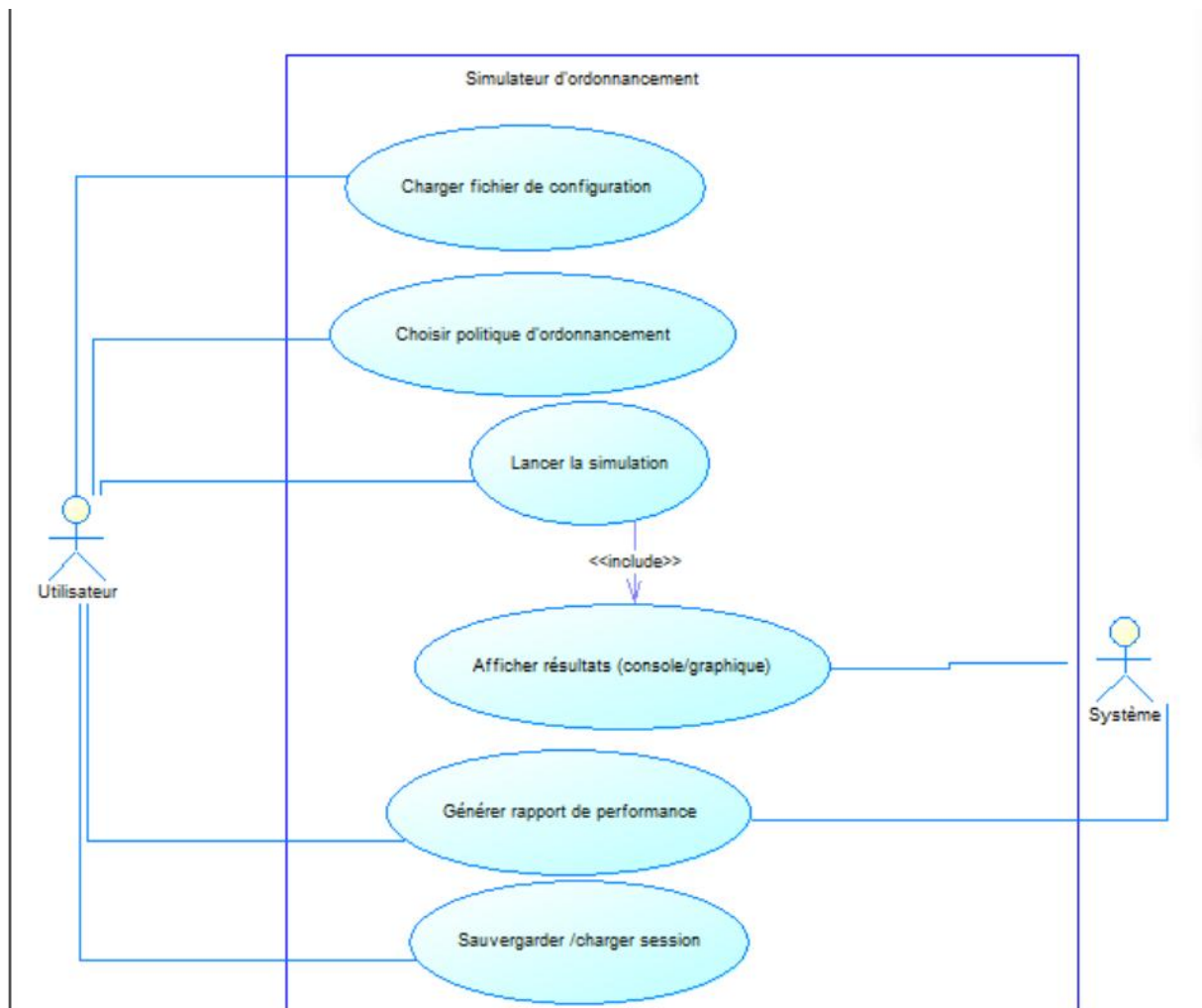
Année académique : 2025-2026

I. Partie 1: Ordonnanceur:



Les diagrammes UML présentés dans ce document ont été conçus et modélisés à l'aide de l'outil PowerAMC, un environnement de modélisation reconnu qui nous a permis de produire des représentations structurées et conformes aux standards UML 2.0.

1. Use case :

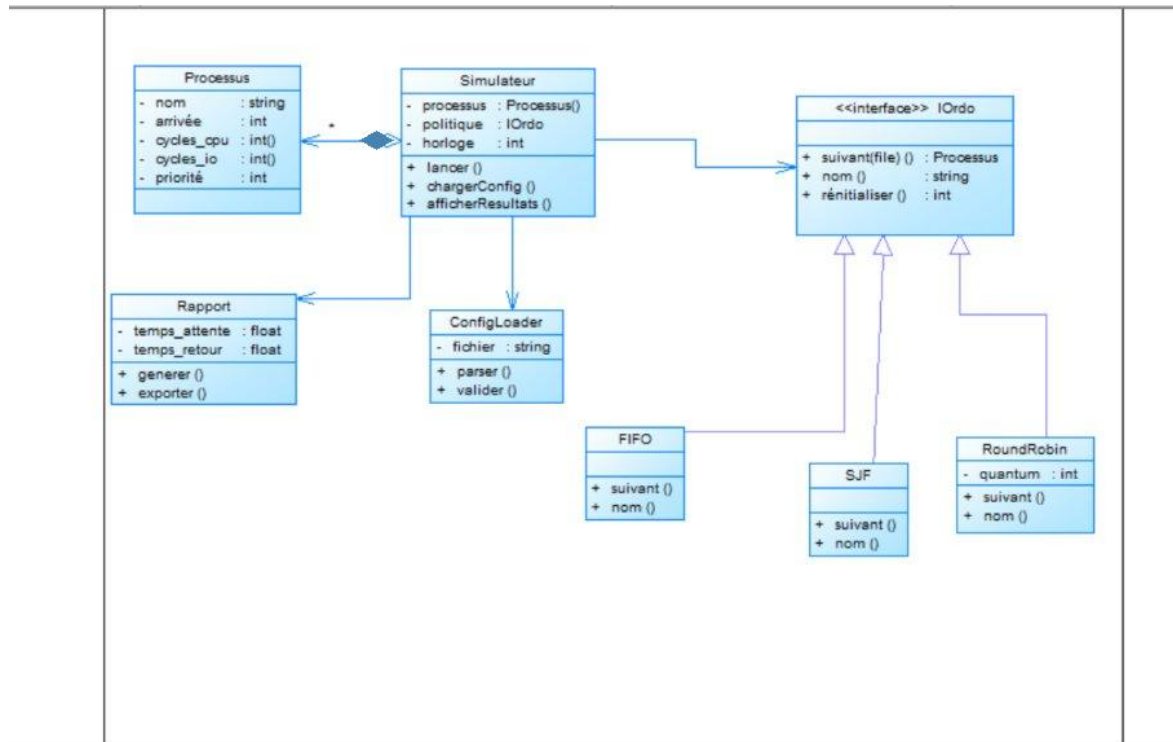


Explication:

- Le diagramme de cas d'utilisation du simulateur d'ordonnancement présente les interactions entre les deux acteurs du système et les fonctionnalités offertes. L'acteur principal, **l'Utilisateur**, interagit directement avec la majorité des cas d'utilisation : il commence par **charger un fichier de configuration** décrivant l'ensemble des processus à simuler, puis **choisit la politique d'ordonnancement** souhaitée (FIFO, SJF, Round-Robin, etc.) parmi celles disponibles dynamiquement dans le répertoire dédié. Il lance ensuite la **simulation**, qui inclut automatiquement via une relation <<include>> **l'affichage des résultats** en mode console ou graphique. L'utilisateur peut également demander la **génération d'un rapport de performance** comparant les différentes politiques, ainsi que la **sauvegarde ou le chargement d'une session** de simulation pour une reprise ultérieure. Le second acteur, le **Système**, intervient uniquement au niveau de l'affichage des résultats, car c'est lui qui exécute la simulation en arrière-

plan et produit les données à visualiser. La relation <<include>> entre "Lancer la simulation" et "Afficher résultats" traduit le fait que tout lancement de simulation déclenche obligatoirement l'affichage des résultats ce n'est pas optionnel mais une étape systématique du processus.

2. Diagramme de classe :



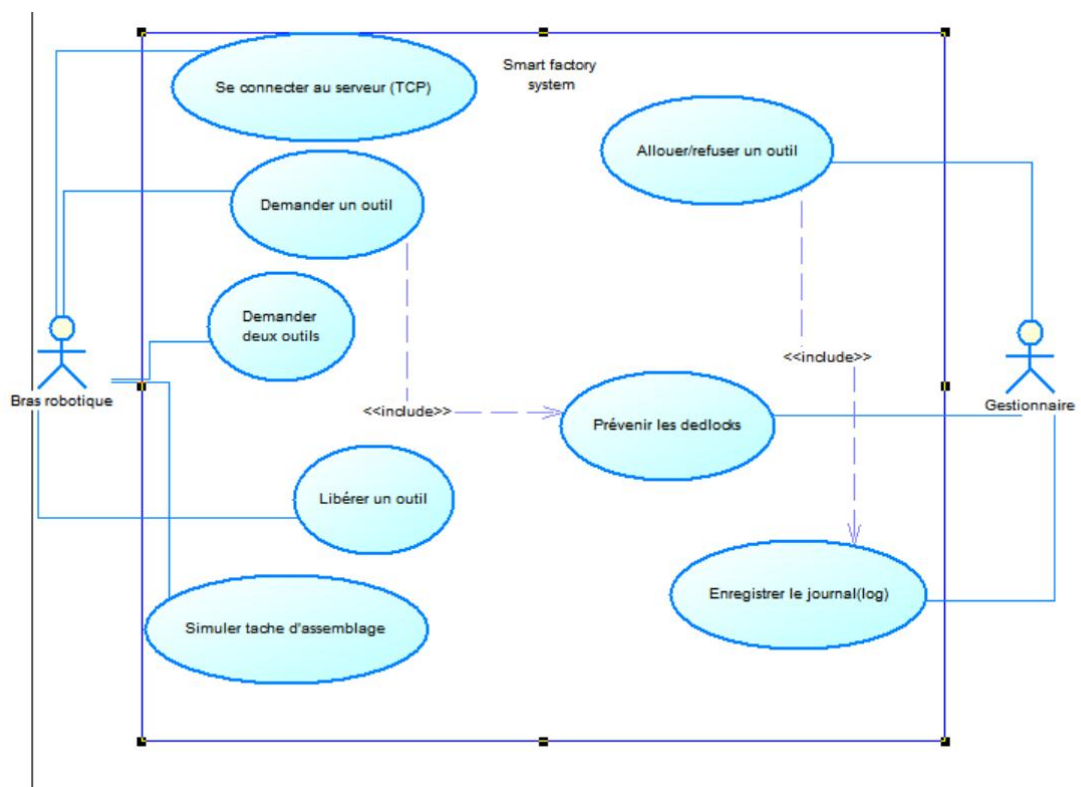
Explication:

- Le diagramme de classes du simulateur d'ordonnancement décrit la structure statique du système et les relations entre ses composants. La classe centrale **Simulateur** orchestre l'ensemble du fonctionnement : elle maintient une collection de **Processus** (relation de composition avec multiplicité *), chacun défini par son nom, sa date d'arrivée, ses cycles CPU et I/O ainsi que sa priorité. Le Simulateur délègue la lecture du fichier de configuration à la classe **ConfigLoader**, qui se charge de parser (ou *analyser syntaxiquement*) et valider le fichier texte passé en paramètre. Pour l'ordonnancement, le Simulateur utilise l'interface **IOrdo** (abstraite) qui

définit un contrat commun avec trois méthodes : suivant() pour sélectionner le prochain processus, nom() pour identifier la politique, et réinitialiser(). Cette interface est implémentée par trois classes concrètes (héritage) **FIFO** (premier arrivé, premier servi), **SJF** (plus court job en premier) et Round Robin (qui ajoute un attribut quantum pour définir la durée de chaque tranche de temps) permettant ainsi d'ajouter de nouvelles politiques sans modifier le Simulateur (principe ouvert/fermé). Enfin, le Simulateur produit un objet **Rapport** contenant les métriques de performance clés, notamment le temps d'attente moyen et le temps de retour, avec des méthodes pour générer et exporter ces statistiques. L'ensemble de cette architecture repose sur le **patron de conception Stratégie**, où la politique d'ordonnancement est interchangeable dynamiquement à l'exécution.

II. Partie 2: Entrepôt intelligent:

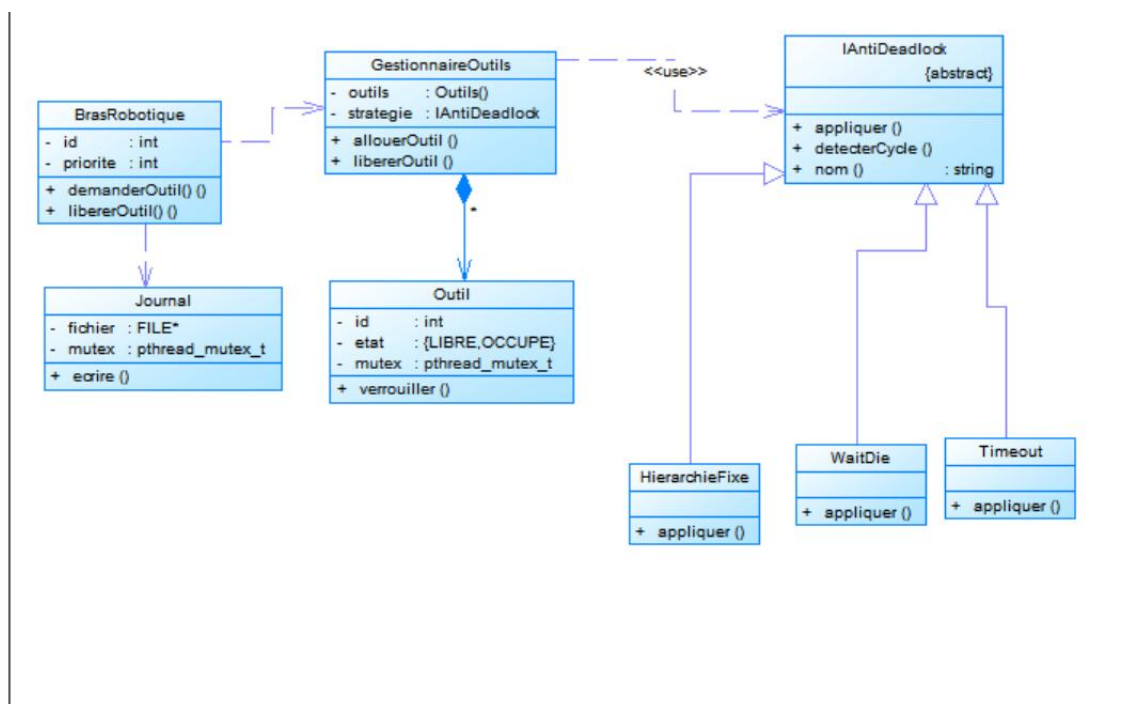
1. Use case :



Explication:

- Le diagramme de cas d'utilisation du système **Smart Factory** met en évidence les interactions entre les deux acteurs principaux et les fonctionnalités du système de gestion distribuée de ressources. L'acteur **Bras Robotique**, représentant chaque client multithreadé, dispose de cinq cas d'utilisation : il commence par **se connecter au serveur via TCP**, puis peut **demandeur un outil** ou **demandeur deux outils simultanément** avant de démarrer son travail, **libérer un outil** une fois la tâche terminée, et enfin **simuler une tâche d'assemblage** durant laquelle il utilise les outils alloués. L'acteur **Gestionnaire**, représentant le serveur central, prend en charge deux cas d'utilisation côté serveur : **allouer ou refuser un outil** selon sa disponibilité, et **enregistrer le journal des actions** avec horodatage. Le diagramme fait apparaître deux relations <<include>> importantes : d'une part, toute demande d'outil inclut obligatoirement l'étape **Prévenir les deadlocks**, garantissant qu'aucune situation de blocage mutuel ne peut survenir entre les bras ; d'autre part, l'allocation d'un outil inclut systématiquement l'**enregistrement dans le journal**, assurant une traçabilité complète de toutes les opérations du système. Ces relations <<include>> traduisent des comportements non optionnels qui s'exécutent automatiquement à chaque fois que le cas d'utilisation parent est déclenché.

2. Diagramme de classe :



Explication:

-
- Le diagramme de classes de la Partie 2 décrit l'architecture orientée objet du système Smart Factory. La classe **BrasRobotique** représente chaque client du système, identifié par un id et une priorité, et expose deux méthodes principales : `demandeOutil()` et `libererOutil()`. Elle est associée à la classe **Journal** qui assure la traçabilité de toutes les actions via la méthode `ecrire()`, en s'appuyant sur un `pthread_mutex_t` pour garantir des écritures thread-safe. La classe centrale **GestionnaireOutils** joue le rôle de serveur : elle maintient une collection d'objets **Outil** (relation de composition symbolisée par le losange plein), chaque outil étant caractérisé par son id, son état {LIBRE, OCCUPE} et un mutex pour protéger les accès concurrents via la méthode `verrouiller()`. Le **GestionnaireOutils** utilise — via une relation `<<use>>` la classe abstraite **IAntiDeadlock**, qui définit un contrat commun avec trois méthodes : `appliquer()`, `detecterCycle()` et `nom()`. Cette interface est héritée par trois classes concrètes implémentant chacune une stratégie de prévention des interblocages : **HierarchieFixe** (ordre fixe d'acquisition des outils), **WaitDie** (protocole basé sur l'ancienneté des processus) et **Timeout** (délai d'attente maximum avant réessai). Cette architecture repose sur le **patron de conception Stratégie**, permettant de changer dynamiquement la politique anti-deadlock sans modifier le reste du système.

III. Conclusion:

La modélisation UML réalisée avec **PowerAMC** nous a permis de concevoir une représentation claire et structurée des deux systèmes étudiés. À travers les diagrammes de cas d'utilisation, nous avons identifié les acteurs principaux de chaque partie ainsi que leurs interactions avec le système, mettant en évidence les fonctionnalités essentielles et les relations entre elles, notamment les dépendances `<<include>>` qui traduisent les comportements obligatoires du système.

Les diagrammes de classes ont quant à eux révélé l'architecture orientée objet des deux parties, en exposant les attributs, les méthodes et les relations entre les différentes classes, qu'il s'agisse de compositions, de dépendances ou de réalisations d'interfaces. Ces diagrammes ont également mis en lumière l'utilisation du patron de conception Stratégie, aussi bien pour les politiques d'ordonnancement que pour les mécanismes de prévention des interblocages.

Cette démarche de modélisation en amont nous a appris qu'une bonne conception précède toujours une bonne implémentation, et que les diagrammes UML constituent un langage universel indispensable pour communiquer, documenter et valider une architecture logicielle avant tout développement.

« Le savoir est la seule richesse qui s'accroît quand on la partage. »

Fin